Khảo Sát Chuyên Sâu: Thuật Toán & Chuỗi Ống Dữ Liệu (The Data Pipeline)

Vì lõi của một hệ thống **Web Crawler** không nằm ở vẻ đẹp UI, mà nằm ở sức mạnh của cỗ máy Nhai và Nuốt Dữ liệu. Tài liệu này cung cấp cái nhìn siêu chi tiết (Low-level) vào vòng đời của một bài báo, từ lúc nó là thẻ HTML, đến khi nó nằm vĩnh viễn trên Amazon S3 Data Lake.

Luồng Dữ liệu (Data Pipeline) được phân rã ra làm **5 Giai Đoạn (Phases)**.

Phase 1: Màng Lọc & Chuẩn Hóa Dữ Liệu (The Normalization Phase)

Crawler không chỉ tải HTML xuống, nó liên tục phải dọn dẹp "Rác". Dữ liệu thô từ Internet luôn lộn xộn.

1. **Chuẩn hóa Đường dẫn (Canonical URL Normalization)**:
   - Một bài báo thường có hàng chục dạng Link:
     - `https://baochinhphu.vn/tin-tuc.html`
     - `http://baochinhphu.vn/tin-tuc.html?utm_source=facebook&utm_medium=cpc`
   - **Cách xử lý**: Worker chạy tệp `normalize.py`. Bóc chóp các giao thức (Force về `https`), loại bỏ sạch sẽ đuôi Query Parameters (`?utm=...`) vô nghĩa. Điều này cốt lõi để thu được 1 chuỗi **Canonical URL** duy nhất. Nếu không làm vậy, DB sẽ chứa hàng triệu bài báo TRÙNG LẶP nội dung nhưng khác URL, làm cạn kiệt dung lượng đĩa.
2. **Batching Payload (Gom Lô Tiết Kiệm)**:
   - SQS cho phép Payload tối đa `256 KB`. Worker KHÔNG BAO GIỜ gửi từng bài báo 1 lên SQS (rất tốn tiền AWS API Call).
   - Worker nhồi 20-50 bài báo thành một danh sách (JSON Array), biến nó thành 1 SQS Message duy nhất.

Phase 2: Hàng Đợi Chịu Tải Cao (The SQS Queue Phase)

Dữ liệu chui vào hàng đợi Amazon SQS. Tại sao lại là SQS thay vì hệ thống Streaming ngầu hơn như Kafka / Kinesis?

- **Logic**: Crawler đi thu bài báo - các bài báo này sống rời rạc, độc lập (Discrete Event), không phải chuỗi luồng click (Stream).
- **Cơ chế Retry Không Rủi Ro (Visibility Timeout)**:
  - Khi Ingester (Lambda) lấy bó dữ liệu 50 bài báo ra khỏi SQS, SQS **không xóa ngay**. SQS kích hoạt còi báo giờ `Visibility Timeout` (Ví dụ 1 phút). Bó tin đó tạm "tàng hình" với Ingester khác.
  - Nếu Lambda nhay bị lỗi (Sập mạng, lỗi RAM), nó không báo Success về SQS. Hết 1 phút, bó tin tự động "hiện nguyên hình" trở lại SQS. Một Lambda Ingester thứ 2 sẽ nhặt lại bó tin đó và thử INSERT (Ghi) lại. Dữ liệu KHÔNG BAO GIỜ BỊ MẤT (At-least-once delivery).
- **Bãi Rác Lọc Lỗi (Dead Letter Queue - DLQ)**:
  - Nếu báo chứa ngôn ngữ Ả rập/Emoji bị lỗi Encoding UTF-8, dội vào DB văng Exception. Lambda chết 3 lần liên tục. Lúc này SQS tống bó tin sang góc gọi là **DLQ**. Admin có thể chui vào DLQ tải file ra xem lại sao lỗi mà không làm kẹt hàng đợi hệ thống.

Phase 3: Cơ Chế Bơm Máu Nuốt Lô (The Ingestion Phase)

Lớp Serverless Lambda dùng `pg8000` đóng vai trò là Lưỡi Dao Bơm Dữ Liệu vào Database lõi.

1. **Tạo Pool Connection Đỉnh Cao**: Khi Lambda khởi động rọi thẳng vào RDS Master, nó không mở 50 connect đứt quãng. Nó mở **1 Connection Transaction TCP**.
2. **Chiến lược "Idempotent Bulk Insert"**:
   - Lambda giải nén cái JSON 50 bài và ném vào DB chung 1 Transaction (Execute Many).
   - Nếu trong 50 bài đó có 10 bài đã được Crawl ngày hôm qua?
   - **Xử lý**: Lỗi Trùng Lặp (Duplicate). Nhưng thay vì báo sập Transaction, Câu lệnh SQL được thiết kế kiểu `INSERT INTO ... ON CONFLICT (canonical_url) DO NOTHING` (Hoặc bắt try/catch bỏ qua). Transaction đi trơn tru nuốt 40 bài báo mới vào dĩa, vất bỏ 10 bài báo cũ tại ngay bộ nhớ CSDL.
   - *Tại sao không SELECT kiểm tra trước cho chắc?*: Câu lệnh `SELECT EXISTS(...)` mất 1 nhịp vòng IO quét DB, sau đó `INSERT` mất nhịp 2. Viết `ON CONFLICT` ép DB tự dùng B-Tree Index bắt lỗi trùng ở cấp độ nhị phân (Low level của ổ cứng C), tốc độ Insert nhanh gấp 2-3 lần.

Phase 4: Thiết Kế Bãi Đáp "Hot Data" (The PostgreSQL RDS Cấu Trúc Động)

Khi chạy Crawler Enterprise, cái làm sập DB đầu tiên là Hệ thống Index chọc ngoáy, không phải RAM.

1. **Đế Chế Indexing (Lập chỉ mục)**:
   - Dòng CSDL bọc 1 `UNIQUE INDEX` lên `canonical_url` để phục vụ ON CONFLICT như trên.
   - Thêm 1 `INDEX` vào `published_at` hoặc `fetched_at`. Vì giao diện Web Dashboard liên tục Request `Select Top 50 ORDER BY fetched_at DESC`. Không có Index này, Table Scan 10 triệu bài báo sẽ mất 20 giây làm chết đơ 100% Web App.
2. **Khái Niệm Partitioning Theo Thời Gian Trục (Table Partitioning)**:
   - Data mới mỗi ngày cực nóng. Data từ tháng trước cực nguội.
   - Khi bảng `articles` chạm 20 triệu rows, câu lệnh Insert bị chậm vì phải đi gõ lại B-Tree Index cho cả 20 triệu bản.
   - Giải pháp **PostgreSQL Table Partition by Range**: Dữ liệu cứ sang tháng mới (bước qua 01/05/2026), Postgres tự mở một phân trang Sector Mới Tinh (`articles_2026_05`). Data mới trút vào file ổ cứng mới, quét Insert và Select chóp siêu tốc độ (O(1) gần như bộ nhớ). Bảng tháng 4 tự nằm im đọc cực lẹ.
3. **Vacuum Cleaner (Hút Bụi Data)**: Postgres Auto-Vacuum bật sẵn. Nó dọn dẹp triệt để các bóng ma bản ghi hỏng từ transaction fail của Crawler để ổ đĩa không phình to.

Phase 5: Hầm Băng Ngàn Năm & Kho Phân Tích (The S3 Data Lake)

Trữ hàng triệu Text/Summary báo điện tử trong một Relational DB 100GB SSD giá rất cao (Khoảng $300-$500/tháng). Amazon S3 100 GB giá chỉ rẻ bằng một ly cafe (Tầm $2.5/tháng). Giải quyết bài toán Tích Trữ Toàn Vũ Trụ:

1. **Cold Data Export (Nén Dữ Liệu Lạnh)**:
   - Giao hệ thống một Cronjob Job tự động chạy ngày 1 hàng tháng lúc 3h sáng.
   - Cron quét Partition của Tháng đó trừ đi 6 Tháng trước (Tức là báo cách đây nửa năm).

- Biến data thành Cấu Trúc **Parquet / JSONL (JSON Lines)**. Đẩy lên S3 bucket.
- Định dạng thư mục S3 như sau: `s3://crawler-bucket/data/year=2026/month=03/articles.parquet`. Định dạng nén thuật toán Snappy giảm kích thước file văn bản xuống chỉ còn 20%! Định dạng Parquet là định dạng lưu trữ dạng cột (Columnar), bỏ qua load cục Text để quét nhanh URL.
- Sau khi xuất thành công, DROP hẳn Partition Bảng Tháng đó trên PostgreSQL RDS. Lúc này RDS mỏng nhẹ, lại chạy siêu mượt hằng ngày.

2. **Kích hoạt Siêu Cỗ Máy Amazon Athena**:
   - Nếu sếp hỏi: *"Team Data, lấy cho tôi biểu đồ số bài tin tức chính phủ từ năm 2024 đến năm 2026 xếp theo Nguồn Mạng!"*
   - Khỏi cần nhập ngược đống file `.parquet` S3 vào hệ thống RDS làm gì mất thời gian.
   - Bật Amazon Athena -> Gõ 1 dòng Query SQL Thần Thánh `SELECT source, COUNT(*) FROM s3_crawler_bucket WHERE year >= 2024 GROUP BY source;`
   - Athena càn quét song song 1 Petabyte ổ cứng đĩa S3 đếm cho bạn trong vỏn vẹn `5 giây` và in luôn Chart lên QuickSight. Bạn chỉ phải trả vài Cent cho cái Request đếm S3 đó.

Kết Luận Mạch Máu Dữ Liệu

Với luồng: **[CRAWLER BATCHING] -> [SQS BUFFER DLQ] -> [LAMBDA UPSERT] -> [POSTGRES PARTITION] -> [S3 DATA LAKE]**, mạng sống của Data không bao giờ chạm Deadlock. Ở mức scale trung bình, nó tiết kiệm và chạy miễn phí mượt mà. Đẩy tới mức scale cực đại, hệ thống chỉ việc gia tăng phần cứng chiều dọc ở RDS và SQS tự động, Data Engineer hoàn toàn thoải mái kê cao gối ngủ.